

Exploration of Rewards Shaping and Immitation Learning in Chess

24.01.2026

Jakob Lambert-Hartmann

Contents

Introduction	2
Background	2
Imitation Learning	2
Reinforcement Learning	2
Properties of Chess	2
Puzzle Data	3
Grandmaster Data	4
Methods	5
Architecture	5
Imitation Pretraining	5
Reward Functions	5
Findings	5

Introduction

In this Report we investigate imitating learning as a form of pretraining. Specifically, we explore the effectiveness of pretraining with imitation learning with heavily biased data. We explore this for the game of chess, because of the availability of both heavily biased data (puzzle datasets), and less biased data (grandmaster games).

To isolate the effect of the bias in pretraining data, we train models with different architectures and reward functions.

Background

Imitation Learning

Reinforcement Learning

Properties of Chess

For an agent to be successful in Chess it needs to master strategies and tactics. **Tactics** defines moves that achieve short term gains. This may be capturing opponent pieces or checkmating the opponent king. **Strategy** on the other hand defines moves with a long term benefit. This could be positioning a piece to put pressure on the opponents king, which may lead to a checkmate in 20-30 moves. See Figure 1 for example positions.

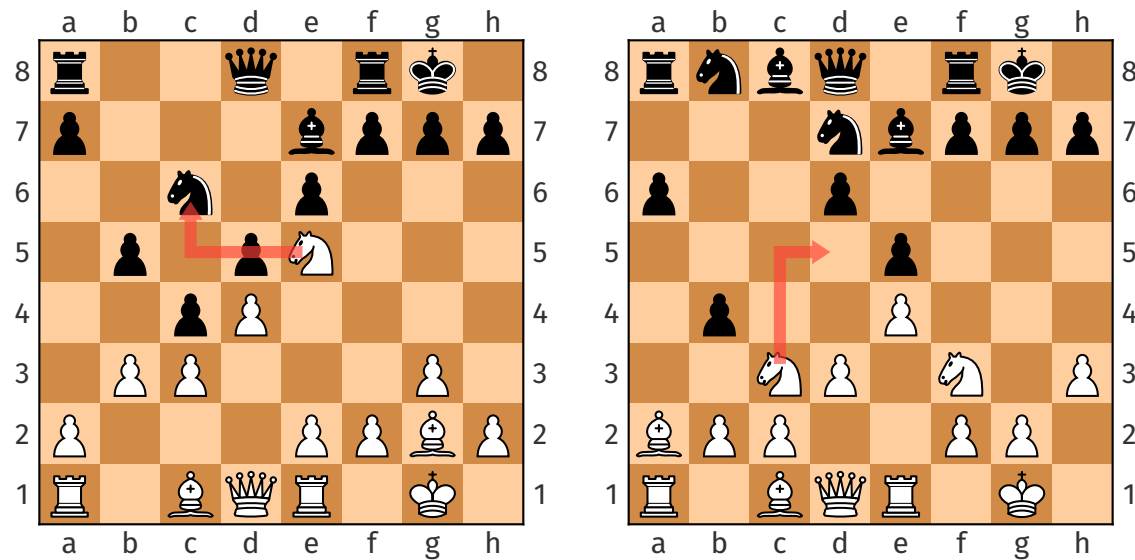


Figure 1: The position on the left has a tactical motif, where the black knight can be captured by white, as it is not defended. On the other hand the right position has a strategic motif, where the knight is attacked and has to retreat. There are multiple available squares, but d5 is the best option. There is no immediate gain, however in the long run the knight will be able to exert more pressure on the opponent from this square.

Puzzle Data

Players that want to improve their tactical perception can train on puzzles. A chess puzzle contains a position with a tactical motif (e.g. checkmate,...). A player then needs to play 4-5 moves to prove their understanding of the motif.

There are many published datasets containing chess puzzles, such as These datasets are highly biased in that they solely contain tactical positions, underrepresenting strategic motifs. Furthermore, positions in such datasets always contain a tactical motif. The lack of negatives may lead the model to be tactically overconfident, leading to false positives (see Figure 2).

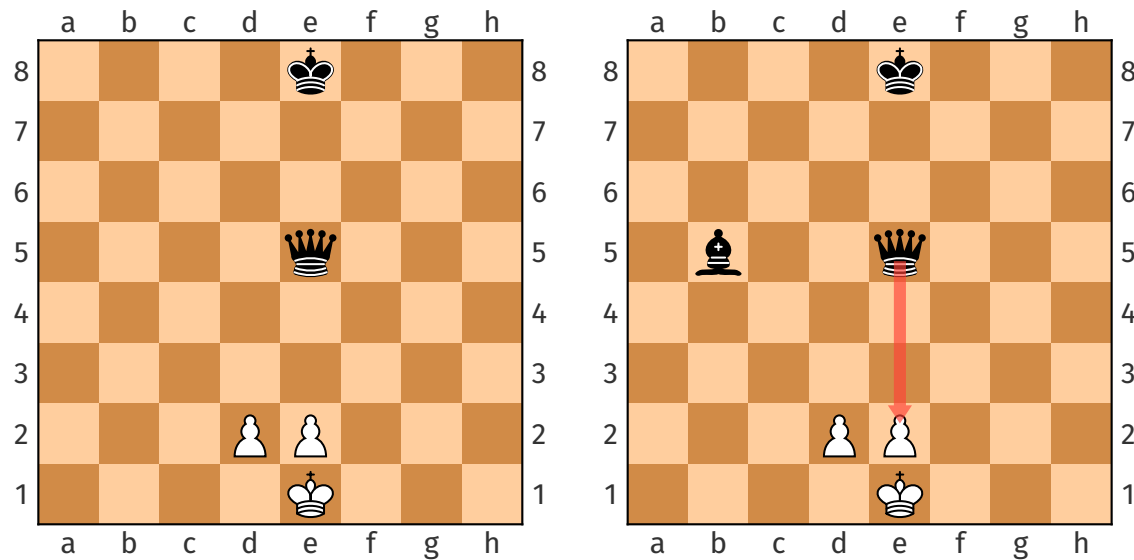


Figure 2: both positions are similar except for the black bishop on d5. In the position with the Bishop (right) black can win the game in one move by checkmating with the queen on e2. playing queen e2 in the other position will loose the game.

Grandmaster Data

Grandmaster games on the other hand are less biased than the puzzle dataset. They cover entire games, and therefore contain positions with tactical motives, and strategic motives

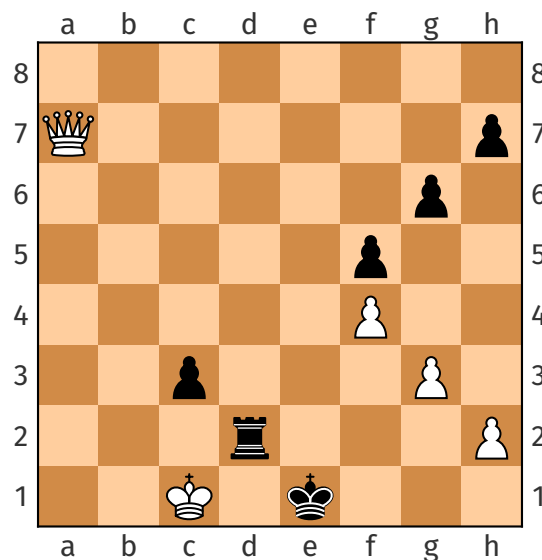


Figure 3: This is the last position of a game by the two former world champions Garry Kasparov and Veselin Topalov. Black resigns in this completely lost position. This is obvious to a grandmaster, in our case however, it will lead to an underrepresentation of checkmates, and will make it difficult for a model to convert completely winning positions.

Methods

Architecture

We train agents with three different architectures: a Feedforward neural network, a Convolutional neural network, and a Resnet. While the networks differ in their hidden layers, they all share a common input and output dimension:

The network **input** contains the current position. The input contains a piece encoding (12 pieces: pawn, rook, knight, bishop, queen, king for both black and white) for each square on the chess board (8×8), resulting in an input tensor of shape $8 \times 8 \times 12$.

The **output** is used to generate a move distribution. It returns two vectors $m_{\text{from}} \in \mathbb{R}^{64 \times 1}$, and $m_{\text{to}} \in \mathbb{R}^{1 \times 64}$. A move matrix is then created from $M = m_{\text{from}} m_{\text{to}} \in \mathbb{R}^{64 \times 64}$. Illegal moves are masked, and move distribution is created.

Imitation Pretraining

Reward Functions

Findings